# A2 — An Imputer class for the centre pixel of a patch

MLR1ILV — Machine Learning 1 · IMC Krems, WS26/27

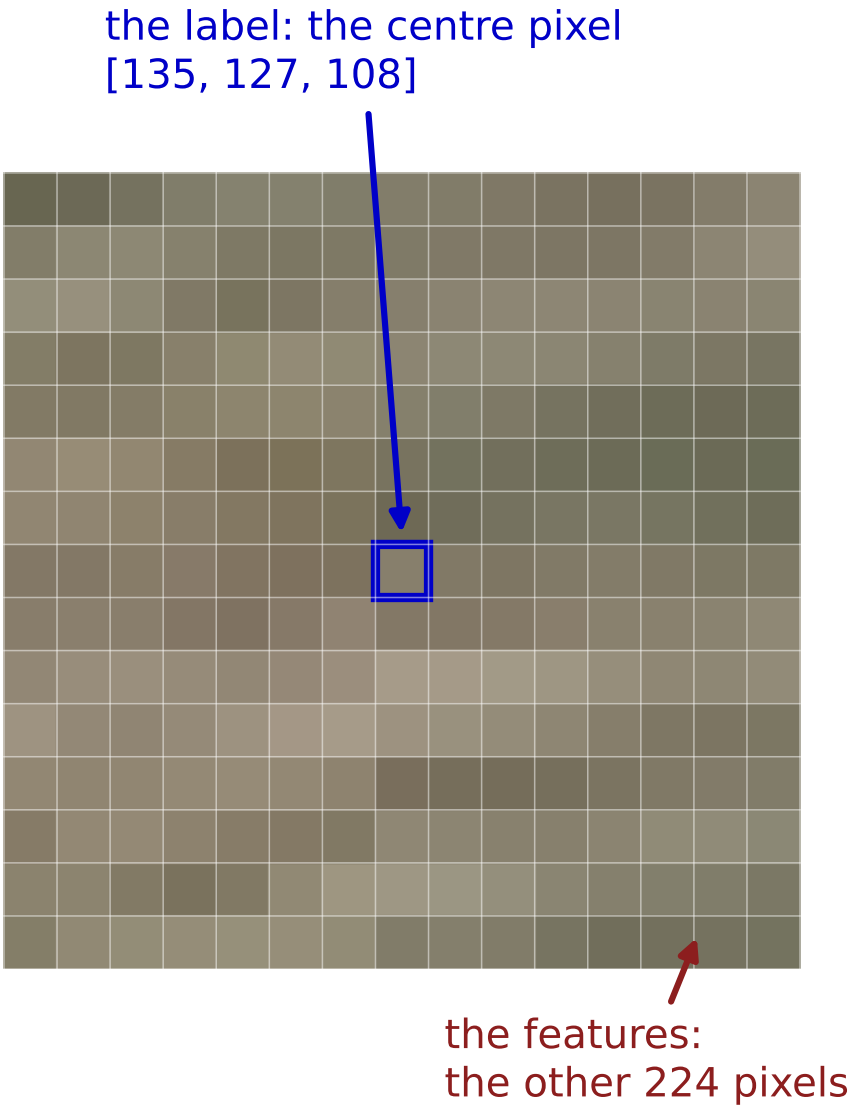
Starts in Session 4 (Fri 09.10, online) · **due Tue 20.10.2026, 23:59** · 100 points

## WHAT THIS IS ABOUT

One pixel of a photograph is missing: a dead sensor cell, a pixel lost in transmission. Your job is an **imputer**, a learned hypothesis that fills it in from the pixels around it.

One data point is a patch of the A1 photograph with an **odd** number of rows and columns. Its **label** is the colour of the centre pixel: three values $(y_R, y_G, y_B)$ between 0 and 255. Its **features** are all the other pixels of the patch, so $d = 3\,(H\,W - 1)$, which is 672 for a $15 \times 15$ patch. Colour values are used as they are, nothing is rescaled.

`A2.py` is attached. It cuts the patches, splits them into a training and a validation set, and carries the class with both methods empty. You fill in the two methods, and nothing else is required. I then import your file, build an `Imputer` of my own, train it on my patches and ask it to predict the centre of patches it has never seen.




One data point: a $15 \times 15$ patch of the Wachau photograph. The framed pixel in the middle is the label, the other 224 pixels are the feature vector. Your patches may be any odd size. (Datenquelle: basemap.at, CC BY 4.0)

## WHAT TO DO

1) **Implement `Imputer.train(patches)`.** `patches` is a `numpy` array of shape $(m, H, W, 3)$: $m$ data points, all of one odd size, values $0 \ldots 255$. **No labels are passed in**: the label of a patch is its own centre pixel, and your method takes it out itself. Everything the trained model needs must be **stored on the object**: the template gives you `self.model`, which may hold anything you like (a fitted `scikit-learn` estimator, an array of weights, a tuple of several things), and you may add further attributes such as `self.patch_side`. Nothing outside the class survives:

when I import your file, no line below the class has run. `train` has to finish within **two minutes on 5,000 patches of** $15 \times 15$.

2) **Implement `Imputer.predict(patch)`.** `patch` is one array of shape $(H, W, 3)$ with $H$ and $W$ **odd**, of *any* odd size, not only the one you trained on. It returns an array of three numbers, the predicted $R$, $G$, $B$ of the centre pixel. If the input is not of that form, or if `train` has not been called, **return the integer** $-1$; the template already has those checks. It must **never read the centre pixel** of what it is given: that pixel is the answer, not a feature, and the test run overwrites it to find out.

**You are done when**

Running `A2.py` ends in **four lines**, and they are the contract the test run depends on. A fresh `Imputer` is trained on the training patches and then asked for four predictions:

```
15 x 15 -> [86.8 93.6 81.9] # three numbers: the size you trained on
 9 x 21 -> [86.8 93.6 81.9] # three numbers: any other odd size too
16 x 16 -> -1 # an even side has no centre pixel
before train -> -1 # predict called before train
```

The first two lines must still be three numbers, and the last two must still be $-1$, once your own model sits in the class; the shape checks that produce the two $-1$s are already written. The three numbers themselves will of course differ from the ones above. The manual computation is answered in the comment block at the bottom of the same file. `scikit-learn` calls it `fit`, not `train`: if you wrap a `LinearRegression` inside your class, your `train` calls its `fit`.

## WHAT YOU HAND IN

One file, **no ZIP**, attached to this Aufgabe in **MS Teams → Aufgaben → A2**, **Abgeben** pressed by **Tue 20.10.2026, 23:59**:

- `A2.py` — your program. Run it with `uv run A2.py`.

It is attached to this Aufgabe as a template: edit it, do not rename it. There is **no why-trail** for this assignment. The template carries everything else you need, including how to get Python running.

Already attached to this Aufgabe, ready to download:

- `get_wachau_photo.py`

## HOW IT IS SCORED

| Part | Points | What it checks |
|---|---|---|
| It runs and the contract holds | 30 | the four lines at the foot of the file print what they should |
| How far your imputer gets below the baseline | 40 | I measure it on my own patches |
| Manual computation, at the bottom of the file | 30 | you understand it yourself |

The baseline is the **average colour of the training pixels**: average the red values of every training pixel, the green values and the blue values, and answer those three numbers for every data point. It needs no training and reaches a validation error of about 1300. Using an LLM to write the program is **expected**. **Code you cannot explain at the five-minute spot-check** is worth nothing, whoever wrote it.

## THE MANUAL COMPUTATION (30 POINTS)

Work this **on paper**. No code, no LLM. Write the answer *and* the working into the comment block at the bottom of your `A2.py`. Part 1 of the written exam asks you to do exactly this, so these six questions are the six practice runs.

### One step of gradient descent, by hand

Linear regression with one feature and one model parameter: the hypothesis is `h(x) = w·x` (a line through the origin, no offset), the loss is the squared error loss. The training set is three days of Krems 2023, `x` the daily minimum and `y` the daily maximum temperature in °C: `(4.2, 11.9)`, `(9.4, 23.9)`, `(8.0, 19.9)`. The current weight is $w^{(0)} = 1.5$ and the learning rate is $\eta = 0.01$.

1) Write the training error `f(w)`, the average squared error loss on the three days, as a formula. Compute the three prediction errors `y − w·x` at `w = 1.5`, and `f(1.5)`.
2) Write the derivative `f'(w)` as a formula and compute `f'(1.5)`.
3) One gradient descent step: $w^{(1)} = w^{(0)} - \eta \cdot f'(w^{(0)})$. Compute $w^{(1)}$ and $f(w^{(1)})$. Did the training error go down?

## DICTIONARY ENTRIES FOR THIS ASSIGNMENT

The entries to read alongside it, from the **Dictionary of Applied Machine Learning** (dictionary-ofml.org). Every key and entry name below is a link to that term's page.

| key | entry | what it settles |
|---|---|---|
| `linreg` | **linear regression** | a linear hypothesis, trained with the squared error loss |
| `mse` | **mean squared error (MSE)** | the squared error loss averaged over a dataset: training error, validation error |
| `gd` | **gradient descent (GD)** | one step against the gradient, scaled by the learning rate |
| `learnrate` | **learning rate** | decides how far the step goes; too large and the training error grows |

Deadline clashing with work or family? Message me in Teams *before* it passes.